Module 6 Tangent: Macros

Macros

Code is data, data is code.

0.

Assignments

So little to do and so much time… wait.

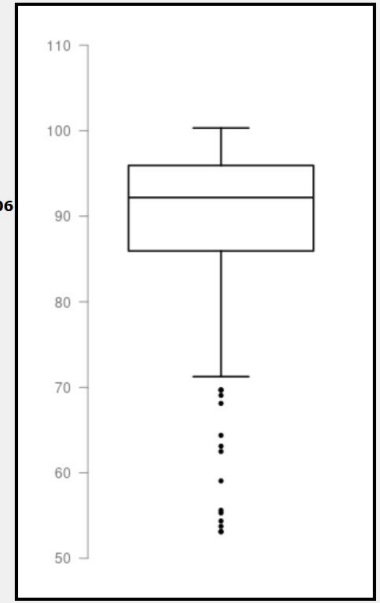
Evaluator Final

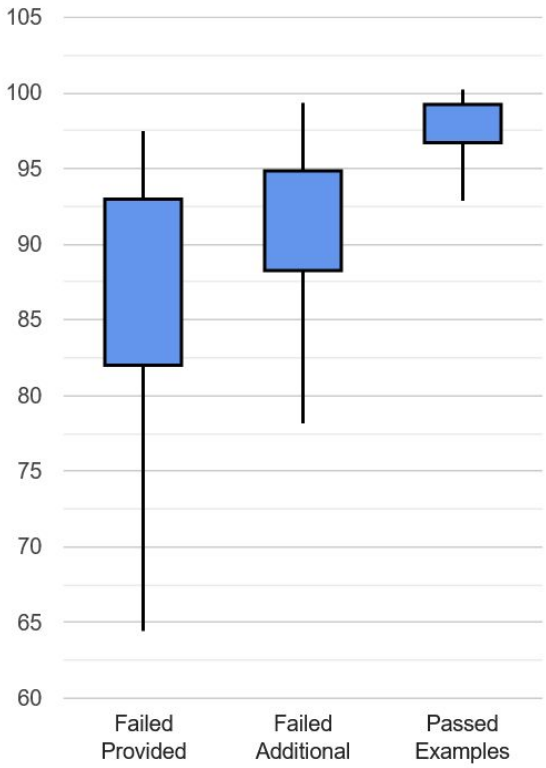
Average 89.8%, Median 92.2%

- Overall good; hardest project part!
- Review for issues & double jeopardy.
- Many compilation failures - check comments for requested changes.
- Opted to remove consideration of EvaluateException.ast from grading.
- Most missed test cases:
  - Static Scoping (?)
  - Evaluation Order, especially errors
  - ObjectExpr, especially Methods

Population size: 370
Median: 92.19
Minimum: 53.12
Maximum: 100.31
First quartile: 85.94
Third quartile: 95.94
Interquartile Range: 10
Outliers: 53.12 53.12 53.75 54.37 55.31 55.62 59.06 62.50 63.12 64.38 68.12 69.06 69.69 69.69

Mean
Geometric mean
Harmonic mean
Median
Mode
Standard Deviation
Variance
Mean absolute deviation
Range
Interquartile range
Quartiles
All dispersion data





Partitioned Evaluator Grades

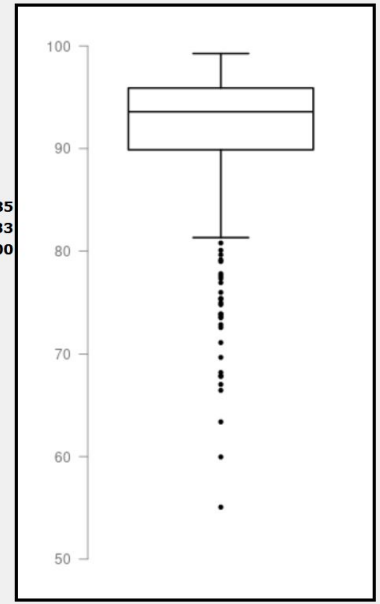
Overall (excl. Exercise 4)

Average 91.4%, Median 93.6%

- Overall strong, left-skewed.
- Most extreme cases involve missing submissions - submit your tests, and definitely submit your assignments.
- Finale is 33% - make sure to go back and fix previous issues!
- If you're struggling with concepts, go to office hours and *ask questions*.

Population size: 376
Median: 93.595
Minimum: 55.07
Maximum: 99.26
First quartile: 89.8825
Third quartile: 95.9
Interquartile Range: 6.0175
Outliers: 55.07 59.96 63.38 66.45 67.03 67.80 67.86 68.19 69.66 71.10 72.57 72.85 73.52 73.62 73.87 73.89 74.80 74.96 75.33 75.41 75.99 76.94 77.34 77.58 77.81 79.00 79.16 79.65 80.09 80.80

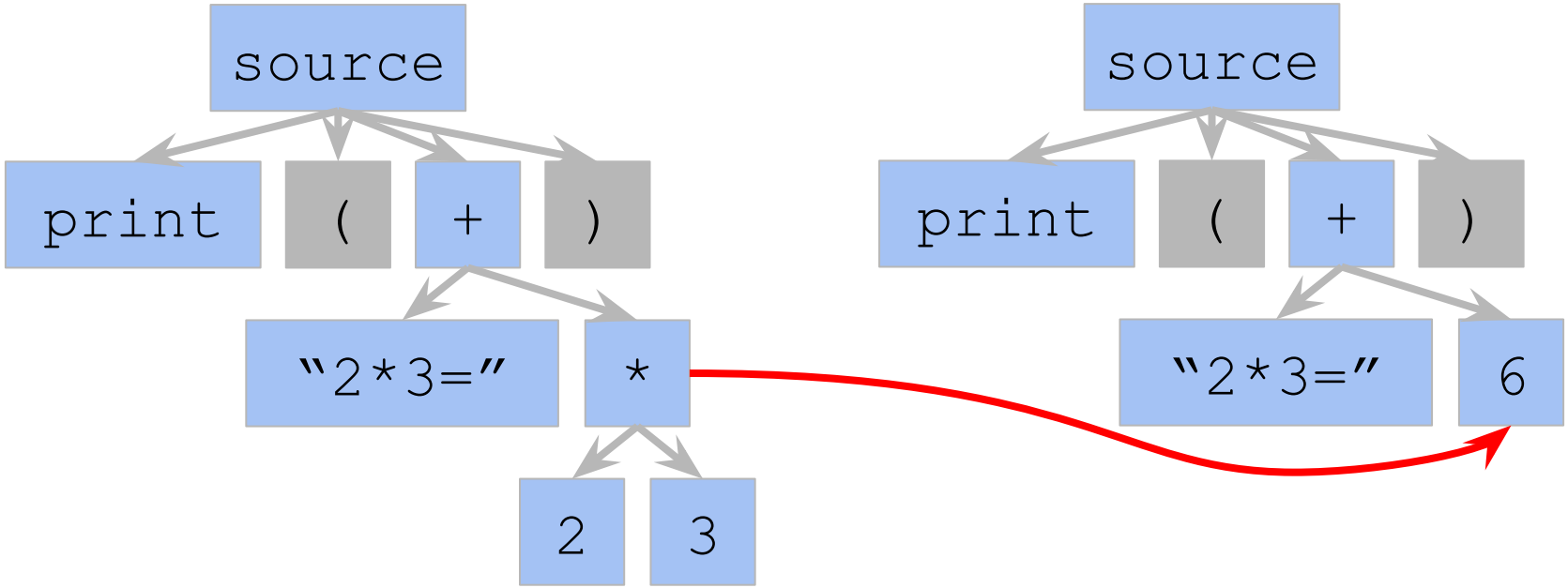
Mean
Geometric mean
Harmonic mean
Median
Mode
Standard Deviation
Variance
Mean absolute deviation
Range
Interquartile range
Quartiles
All dispersion data



1.

Macros

Code is data, data is code.



Code is Data, Data is Code

Macros

Definition

A macro is a function that takes code as input and returns code as output.

- Instead of evaluating code, it is transformed into new output code.
  - May also evaluate at compile-time.
- Lisp languages popularized the idea of “code is data, data is code”.
  - Racket is a language to look at.

class-c-new.rkt (define ...)

```
#lang racket/base
(require "class-internal.rkt"
         "class-c-old.rkt"
         "class-wrapped.rkt"
         "../contract/base.rkt"
         "../contract/combinator.rkt"
         (for-syntax racket/base
                     syntax/name
                     syntax/stx))

(provide class/c2)

(define-syntax (class/c2 stx)
  (define-values (opaque? args)
    (syntax-case stx ()
      [(_ #:opaque args ...)
       (values #t (syntax->list #'(args ...)))]
      [(_ args ...)
       (let ()
         (define stx-args (syntax->list #'(args ...)))
```

```c
#define SQR(n) n * n

int x = 1;
int sqr = SQR(x + 2);
printf("%d\n", sqr); //5..?

int sqr = SQR(x + 2);
        = n * n;
        = x + 2 * x + 2;
        = 1 + 2 * 1 + 2;
        = 5;
```

```c
#define SQR(n) (n) * (n)

int x = 1;
int sqr = SQR(x+2);
printf("%d\n", sqr); //9!

int sqr = SQR(x + 2);
        = (n) * (n);
        = (x + 2) * (x + 2);
        = (1 + 2) * (1 + 2);
        = 9;
```

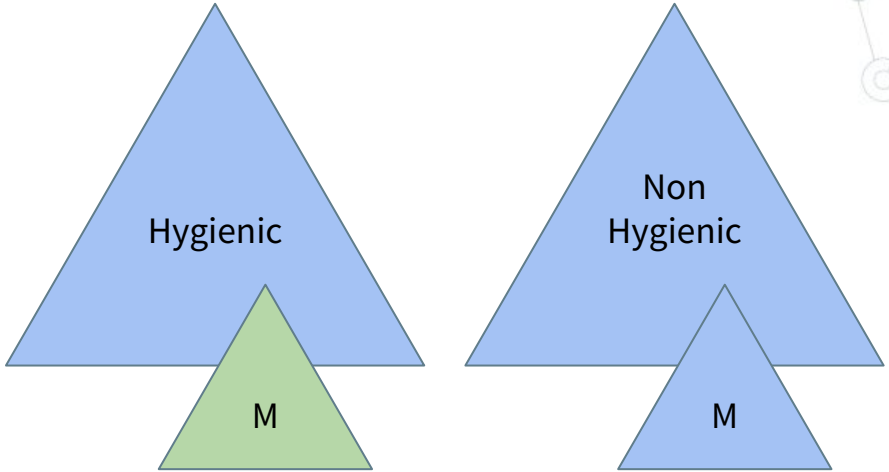
Even C has Macros… Kinda

Hygiene

Definition

Hygiene refers to the scope used to resolve identifiers in macros.

- A *hygienic* macro resolves identifiers at the definition site.
- A *non-hygienic* macro resolves identifiers at the calling site.
  - Similar to dynamic scoping.
- There's a time and place for both, but hygienic macros are safer.



```
//hygienic macro

int x = 1;
macro M() {
  return x;
}

function f() {
  int x = 2;
  return M();
}

f(); //1
```

```
//non-hygienic macro

int x = 1;
macro M() {
  return x;
}

function f() {
  int x = 2;
  return M();
}

f(); //2
```

Macro Hygiene

Module 6: Compilation

Compilation

Time for a Module 6 crash course in one lecture.

Slides Carnival

Free templates for all your presentation needs

For PowerPoint and Google Slides

100% free for personal or commercial use

Ready to use, professional and customizable

Blow your audience away with attractive visuals